The International Journal of Advanced Manufacturing Technology
https://doi.org/10.1007/s00170-025-15830-2

ORIGINAL ARTICLE

Generative AI meets CAD: enhancing engineering design to manufacturing processes with large language models

Amirmohammad Daareyni[1] · Antti Martikkala[1] · Hossein Mokhtarian[1] · Iñigo Flores Ituarte[1]

Received: 22 April 2025 / Accepted: 26 May 2025

Abstract
As a key technology in the field of Artificial Intelligence (AI), large language models (LLMs) have gained substantial attention due to their wide range of capabilities in understanding and generating human-like speech. These advancements extend opportunities beyond text generation offering solutions for more complex engineering tasks. Despite extensive exploration of LLM capabilities in multiple fields, there is a noticeable lack of comprehensive studies within the engineering sector. This study aims to address this gap by investigating the potential of LLMs in solving engineering design problems. This study presents a framework for LLM-assisted CAD that employs advanced artificial intelligence systems, specifically GPT-4o model, to provide users with the ability to generate and manipulate 3D models through text commands, visual inputs such as blueprints, images, voice, and any other type of general or professional information about the desired model. By integrating LLMs with CAD software and incorporating image recognition and voice detection capabilities, we aim to lower the barrier to entry for 3D modeling, making it more user-friendly and accessible to non-professionals. The proposed framework employs LLM to process user inputs (text, images, and/or voice) and generate the code corresponding the CAD model. A closed feedback loop ensures that the generated models are refined iteratively until they meet the user's specifications. This research demonstrates the potential of generative AI-assisted design (GAD) in simplifying complex design processes and providing valuable insights into areas where LLM enhanced processes can assist design to manufacturing workflows.

Keywords Computer-aided design · Design automation · Artificial intelligence · Large language models · Chat-GPT

1 Introduction

AI is reshaping nearly every sector of science, technology, and industry [1, 2]. Within this broad AI landscape, natural language processing (NLP) has emerged as a pivotal field focused on enabling machines to understand, interpret, and generate human language [3]. A significant development in this progression is the rise of LLMs, exemplified by models like GPT-4 developed by OpenAI [4] and BERT developed by Google [5]. Powered by sophisticated transformer-based architectures and massive datasets, these models perform effectively in conventional NLP applications while also extend their capabilities to a variety of domains [6]. Modern LLMs can retrieve information, generate textual and graphical content, write and debug code in multiple programming languages, and assist with complex computations and data visualization [6]. They also facilitate document editing, proofreading, and translation across multiple languages, making them invaluable in fields like education, engineering, creative arts, personal development, and day-to-day life [6]. Leveraging vast amounts of textual data, these models can interpret specialized instructions, identify underlying patterns, and generate structured outputs in line with user-defined objectives [7, 8].

Beyond traditional language-focused tasks, LLMs are increasingly being applied in engineering domains, where

Amirmohammad Daareyni, Antti Martikkala, Hossein Mokhtarian, and Iñigo Flores Ituarte contributed equally to this work

✉ Amirmohammad Daareyni
amirmohammad.daareyni@tuni.fi

Antti Martikkala
antti.martikkala@tuni.fi

Hossein Mokhtarian
hossein.mokhtarian@tuni.fi

Iñigo Flores Ituarte
inigo.floresituarte@tuni.fi

[1] Faculty of Engineering and Natural Sciences, Tampere University, Korkeakoulunkatu 6, 33014 Tampere, Pirkanmaa, Finland

Published online: 05 June 2025

their capacity to interpret structured queries and generate technical outputs can streamline complex workflows. Recent studies have shown that integrating LLMs via accessible Application Programming Interfaces (APIs), such as the ChatGPT API, can facilitate the creation of domain-specific tools that lower technical barriers. For instance, Martikkala et al. [9] developed a system that helps less-experienced users program Arduino-based IoT sensors by integrating ChatGPT with a custom interface. Combining LLMs with engineering templates and pin mappings, the tool enables a semi-automated, human-in-the-loop workflow for accessible prototyping.

Computer-aided design (CAD) has long been a cornerstone of product development, enhancing productivity through efficient visualization, analysis, and documentation while reducing time and cost [10]. It improves design quality by minimizing errors and enabling thorough analysis and exploration of alternatives, while also standardizing documentation and enhancing communication [10]. Over the years, engineers have continually sought to optimize and simplify CAD workflows through various innovations, ranging from parametric and direct modeling to generative design [11–13]. Despite these advancements, however, a significant degree of experience and proficiency remains essential for users to effectively create viable designs in CAD software.

Recent studies have explored the role of AI and machine learning (ML) in optimizing design and manufacturing processes. For example, Careri et al. [14] combined ML and AI to determine the optimal process parameters for a powder bed fusion-laser beam on metal process, aiming to reduce defects and maintain the geometrical and dimensional integrity of the thin features needed for efficient heat transfer in a heat exchanger. Kiangala et al. [15] investigated a new method for human and machine interaction in an Industry 5.0 context by designing a GPT-based industrial bot to enhance worker well-being and production efficiency. The bot monitors production, tracks well-being parameters like pollution levels and overtime hours, and provides functions such as fault detection, root cause analysis, and synthetic data generation.

Similarly, several commercial and non-commercial tools have been developed to make the design process faster and easier by adding AI tools and technologies. For instance, ZOO introduced a text-to-CAD open-source prompt interface, which can generate CAD models through textual prompts [16]. Unlike artistic text-to-3D solutions that rely on point clouds, this model employs boundary representation (B-Rep) to produce fully editable STEP files. As a result, designers can import these files into existing CAD software for precise modifications, utilizing essential geometry and topology data [16]. Deng et al. [17] explored how the integration of LLMs in the design process can reduce the human workload in design process. They used ChatGPT for required file analysis, design computation, and 3D modeling. They propose a framework to connect design requirements, analysis, engineering computing, and model creation with LLM for automated workflow. To show the framework capabilities, they chose a one-stage reduction gear system. In a separate study, Li et al. [18] presented LLM4CAD, a framework enabling GPT-4 and GPT-4v to generate CAD models. They performed a quantitative assessment to measure its effectiveness in conceptual design, focusing on two core capabilities: producing syntax error-free CAD models and creating shapes that closely approximate ground truth geometry. To evaluate these capabilities, they introduced two key metrics: parsing rate, which measures how often the generated code can be parsed successfully, and intersection over union, which compares the overlap between the generated shape and the ground truth shape relative to their total area. Jones et al. [19] propose AIDL (AI design language), a solver-aided domain-specific language that enhances CAD modeling with LLMs by offloading precise geometric computations to a constraint solver. This allows LLMs to focus on high-level reasoning, leveraging their strength in semantic understanding and natural language manipulation. AIDL supports implicit geometry dependencies, explicit constraints, and a hierarchical design structure to improve modularity and editability. In few-shot tests, AIDL outperforms existing tools by generating designs that better match input prompts and are easier to refine, representing a notable step forward in LLM-driven CAD design. In another study, Li et al. [20] introduce CAD-Llama, a framework designed to adapt large language models for parametric 3D CAD generation. By leveraging a structured code format (SPCC) and a hierarchical annotation pipeline, the authors align LLMs with CAD-specific tasks through adaptive pre-training and instruction tuning. The approach demonstrates notable improvements in accuracy and complexity over previous methods.

Other studies attempted to integrate LLMs with CAD tools [17, 18]. However, most of these studies have targeted narrow application domains, lacked a dedicated user interface (UI) for their frameworks, effective feedback loops to ensure the quality of the generated CAD, and ability to process different modalities. To address this gap, this study centers on developing a streamlined API to interact with LLMs, and establishing feedback loops for CAD syntax quality check, refine and visualize the design, and final verification whether the obtained design meets user's query. The specific objectives of this research include: (i) exploring methods to translate multimodal voice, textual, and visual inputs (e.g., sketches, blueprints) into error free CAD models, (ii) developing a three layers closed-loop feedback mechanism wherein the system iteratively refines its outputs based on user feedback or automated validations, and finally (iii) evaluating the accuracy and reliability of LLM-assisted CAD modeling for meeting user-defined specifications using the design to manufacturing of gears as a case study.

Concretely, the current paper proposes GAD, a Python-based application built with Streamlit, providing an interactive environment that allows the users to submit design requirements or queries. These inputs are then processed by the OpenAI GPT API to generate CAD model scripts, which are displayed in real-time for quick iteration without the need for extensive web development. Building on this technical approach, the present study examines how LLMs can simplify and enhance CAD workflows in mechanical and industrial engineering. By identifying the strengths and limitations of this LLM-driven approach to CAD modeling, we aim to lay the groundwork for broader, more accessible design-to-manufacturing workflows that leverage state-of-the-art AI solutions. In addition to its technical advantages, the GAD framework can support evolving educational practices in mechanical engineering. As an open-source platform, GAD offers unrestricted access to AI-driven design tools, enabling collaboration among students, educators, and researchers to enhance its capabilities. By presenting learners with a user-friendly interface driven by AI-guided suggestions, real-time feedback, and rapid prototyping capabilities, GAD facilitates engaging and interactive training experiences that integrate theoretical knowledge with hands-on applications. Consequently, it helps prepare the next generation of manufacturing engineers by fostering both theoretical understanding and practical skills through an AI-enhanced, hands-on learning experience.

2 Methodology

The research methodology for this study investigates the integration of large language models (LLMs) into CAD workflows. The proposed framework aims to employ LLMs for creating accurate and scalable CAD solutions addressing current challenges in CAD design, particularly their inaccessibility to non-experts and steep learning rate. An intuitive UI is developed to streamline interactions, enabling users to provide design requirements and allow the users to review generated models and provide feedback for further refinements.

The system integrates an LLM with OpenSCAD and computer vision techniques to analyze visual inputs, facilitating the generation of models with defined dimensions and interrelated components. Figure 1 represents the overall workflow, starting with uploading a user input query to the LLM. Three feedback loops, including the syntax quality check loop, refine and LLMs' self-evaluation loop, and user feedback loop, ensure the quality of the generated models. Generated model then can be extracted in different formats. Following

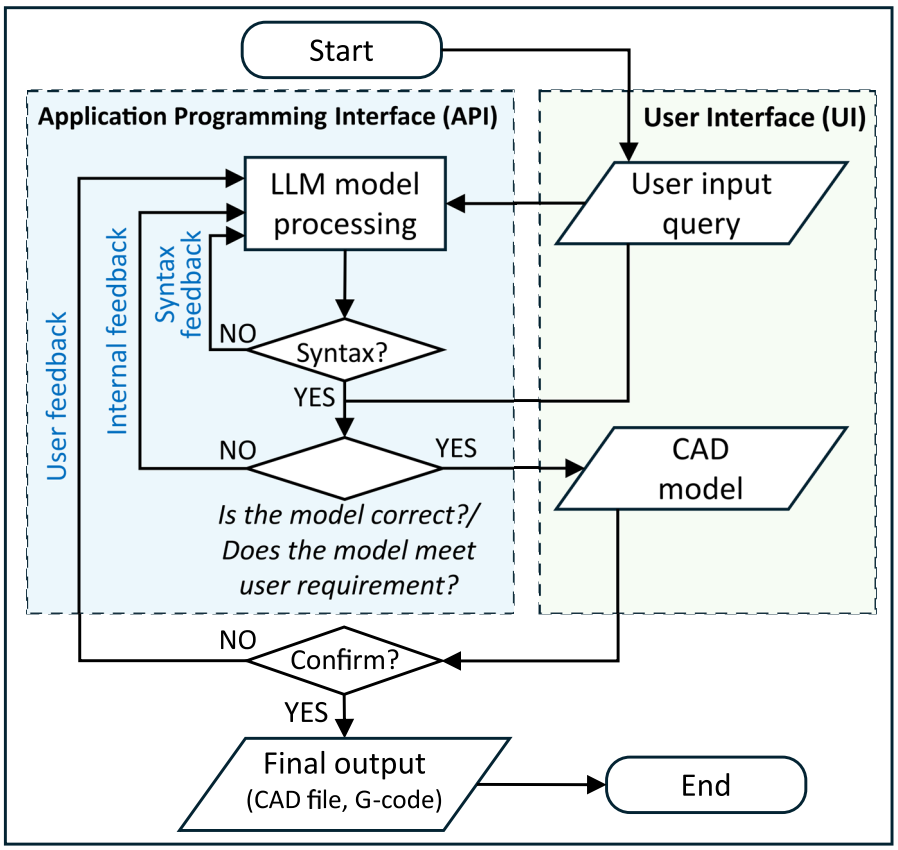

Fig. 1 Workflow for generative AI-assisted CAD modeling (GAD)

is a detailed description of the system architecture along with prompt engineering details, the CAD generation process, and testing and validation techniques.

2.1 System architecture

GAD is structured around three primary layers: the user interface layer, the processing and LLM layer, and the rendering and export layer. This section focuses on explaining the utilized tools and approaches in every layer. Figure 1 demonstrates the overall data flow among GAD components.

2.1.1 GAD user interface (UI)

To streamline interactions among the various tools integrated into GAD, the entire application, including its UI, is built in Python. We use Streamlit open-source Python framework for data scientists and AI/ML engineers as an interactive interface which runs locally but is accessible through a browser [21]. Figure 2 illustrates the GAD UI developed in this study. Figure 2a shows how the UI allows the user to submit textual, audio, and visual inputs to the system. A system log is also displayed to monitor current state of the system (see Fig. 2b). The GAD UI also allows the user to test and utilize different LLM models (see Fig. 2c). Figure 2d shows how the user can activate and control the feedback loops. The GAD system also allows to include external resources for example CAD model libraries to enhance the accuracy of the outcome (see Fig. 2e). The GAD process is connected to several outputs that allow to display the CAD model after completion, generate and save an.stl file, and generate G-code (see Fig. 2f). Once the user inputs are provided, generating the CAD is triggered in Fig. 1g. Additionally, Fig. 2h shows extra functionalities to connect the UI to a 3D printer and trigger the printing process and manufacturing process.

2.1.2 Processing and the LLM layer

The control logic within GAD collects all user's inputs and constructs a structured prompt for the LLM component. This prompt includes further description about the engine and its purpose, expected format for the outputs, and the type and format of inputs provided by the user. Then, this prompt will be sent to GPT API, upon receiving the answer. GAD will parse the response and extract the required information including additional information provided by LLM about the generated model, and the 3D model script. OpenSCAD, an open-source script-based solid 3D CAD software [22], is used as the reference CAD tool for the generation of 3D model script. The generated script then will go through two layers of validation. In the first layer, system will run the SCAD file and ensures that the file runs without any syntax error. Then, as LLMs, in this case GPT-4o, is still not capable of handling 3D models directly, an algorithm creates six projection views of the generated 3D model. Generated projections then will be sent back to LLM along with all of user

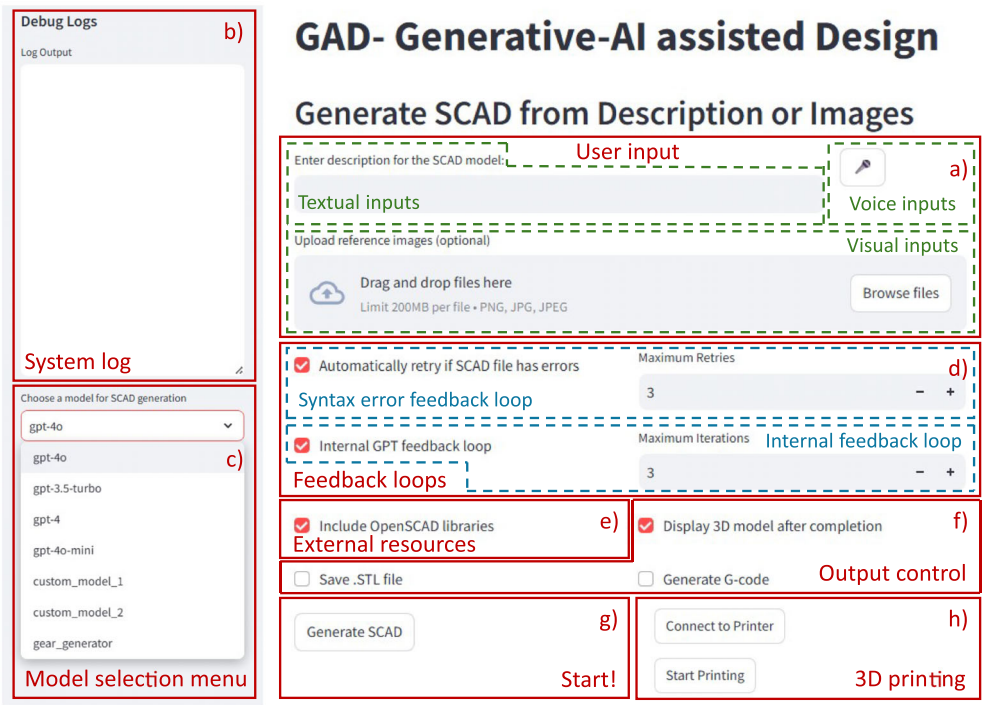

Fig. 2 User interface of generative AI-assisted design (GAD)

inputs, and current CAD script asking the model for evaluating itself and verifying that the provided outputs meet the user request. This internal feedback system ensures that GAD satisfied the request.

Prompt engineering and well-structured prompts are the key to interacting with LLMs (here GPT-4o). Prompts used in this study start with a brief introduction of the role that LLM has followed by user inputs, expected task, and response format. To avoid having unnecessary outputs and to make handling the responses received from LLM easier, we decided to use JSON format for the output. In this study, we used two kinds of prompts: one for the first request and the other one for the internal feedback loop. Table 1 represents two examples of prompts used in this study for creating a model based on descriptions provided by the user using OpenSCAD external libraries, and another one for the internal feedback loop for the same model.

LLMs lack the ability to process audio directly; therefore, a real-time Python-based speech recognition library [23] is used to convert all audio inputs into text, which is subsequently used in the GAD process. When the user uploading images, Base64 encoding converts binary data (i.e., images, files, or other raw data) into a text string. An extra functionality is present in GAD to support uploading images without description; in this case, the following prompt will be sent to LLM asking for descriptions.

"Analyze the following image(s) encoded_image and suggest a descriptive name for a 3D SCAD model. Do not add material, standard, or size-related descriptions if they are not clearly given in my inputs. Replace any invalid characters with underscores (_)."

2.1.3 Rendering and export layer

After verification and validation, OpenSCAD is used for rendering and extracting STL file. In the next step, Slic3r, an open-source slicer engine [24], is used to generate the G-code from 3D CAD files (STL or OBJ). Once this step is complete, an appropriate G-code file to print the 3D modeled part will be extracted. In this paper, Three.js which is an interactive 3D visualization tool is used to display and interact with STL files in UI.

Model generation and rendering is composed of two main tasks, (i) generating the requested part based on user inputs (main loop) and (ii) internal feedback loop to ensure the syntax quality of generated model along with several subtasks to make the LLM response more accurate and reliable. After user enters inputs in form of descriptions or images, the algorithm forms the prompt to support the generating CAD script in the main loop. This prompt will be uploaded to the LLM, and the code will be extracted from the response. The extracted code then will be run in OpenSCAD searching for syntax error in the code. In case of any errors, the algorithm will repeat the process for debugging. User can set the number of retries by changing maximum retries. If LLM cannot create any correct models, the algorithm will be closed with an error and user needs to try again with more detailed inputs.

Table 1 Prompt structure for first quest and internal feedback loop

First quest	Internal loop
"You are a 3D SCAD model generator. Based on the user's description, feedback, and provided reference images or projections, regenerate a valid OpenSCAD (.scad) file. The SCAD code must always be complete, valid, and formatted in OpenSCAD syntax. Use this description to guide the SCAD model generation: {description}. Generate a valid OpenSCAD (.scad) file. Analyze the following OpenSCAD libraries first and use them in this model generation: {combined_libraries}. Do not include explanations, questions, or any non-SCAD information. The SCAD code must always be complete and valid. The provided SCAD code should not be inside triple backticks. Respond in JSON format like this: { "description": "<text>", "code": "<code>"} Special characters like newlines (\n) must be escaped as \\n."	"You created a 3D SCAD model based on the user's description and provided reference images or projections. now you need to evaluate yourself. Here is the current SCAD file content: {current_scad}. Analyze the following OpenSCAD libraries first and use them in this model generation: {combined_libraries}. This is the {view_name.lower()} projection of the model generated by you. Use this description to refine the SCAD model generation: {description}. compare this projection with description and reference images. Is the model that you made good enough? Respond in JSON format. If yes, set the 'response' field to 'Yes'. If no, set the 'response' field to 'No' and include a valid OpenSCAD (.scad) code in the 'code' field. Example response: { "response": "Yes"} OR {"response": "No", "code": "<code>"} Special characters like newlines (\n) must be escaped as \\n."

After ensuring that the code runs smoothly and the CAD script is error free, the provided code along with all user inputs and projections of the current CAD model will be sent back to LLM using the internal loop prompt provided in the previous section for internal evaluation (see Table 1). Expected answers in this part are "Yes" and "No," answering the question stated in the prompt "Is the model that you made good enough?" If "Yes," the algorithm will move forward and saves the requested formats while showing the STL file in the UI. In case of "No," algorithm extracts the new code provided by LLM, checks it for syntax error, and repeats the internal loop with the new code. User can control the number of internal loops in the UI. If model is unable create any good models, the algorithm will be closed with a warning and user needs to decide whether to repeat the process or use the latest version. An additional layer to connect CAD design with CAM is added to the GAD functionalities facilitating rapid prototyping. We integrated Printrun [25], an open-source software for controlling 3D printers used to provide real-time interactions and monitoring. Once GAD generates the G-code, it can be sent to the 3D printer through the UI, allowing users to quickly initiate and oversee the printing process.

2.2 Testing and validation

To evaluate the different capabilities of GAD, four different 3D generation tasks were performed. The first task examines the impact of GAD's self-evaluation system (internal feedback loop) on the quality of generated CAD model. The second investigates the capability of the system to create CAD models based on different types of inputs, in this case visual inputs, and the quality of the generated model. The third explores GAD's performance in generating models from highly detailed as well as poorly detailed user inputs. Finally, the fourth task evaluates GAD's design-to-manufacturing workflow using a one-stage reduction gear system as a case study. In this study, we utilize a Prusa MK4 desktop fused filament fabrication (FFF) 3D printer for manufacturing; however, other 3D printers with different specifications can also be used.

To further evaluate the capabilities of the GAD framework and explore alternative processing cores, two mid-sized open-access LLMs, Google Gemini 2.5 Pro Experimental [26] and OlympicCoder [27], were used in place of the original GAD core (e.g., GPT-4o). These models were accessed and executed via OpenRouter [28], a unified API platform that enables interaction with multiple AI models (such as GPT-4, Claude, Mistral, and others) through a single interface. To analyze the performance of these alternative models, multiple attempts were made to design a one-stage reduction gear system using the same prompt as in the fourth task. Success rate and execution time were recorded for each attempt.

3 Results and discussion

Figure 3 presents the results of validation tests performed using GAD. These tests aim to evaluate various aspects of the system, including the internal loop, its response to different types of inputs, and its response to complete and incomplete user request. Figure 3a illustrates the functionality of the internal loop and its importance in the system. In this quest, for the sake of simplicity and to emphasis that GAD can be used by professional and non-professional for engineering as well as general purposes, GAD is asked to create a simple chair. The three design iterations show that GAD can assess an initial CAD model and refine it to address shortcomings from the previous versions. However, it should be noted that this does not guarantee its effectiveness for all use cases or geometries.

Figure 3b demonstrates how GAD responds to different input types. In this case, a picture of a component without descriptions and dimensions is uploaded to GAD. Although The system successfully interpreted the general circular geometry of the component, including the radial pattern of circular holes and the large central cutout, suggesting GAD's capability to extract and replicate overarching geometric features from visual cues, the output CAD model lacks dimensional accuracy and fails to reproduce the smooth, round contour of the outer edge. These inaccuracies highlight a current limitation of GPT-4o: while it can infer general form and layout from an image, it struggles with precise scaling and complex edge features in the absence of explicit dimensional data. Figure 3 c and d illustrate how LLMs can be extremely relevant in engineering design workflows. In this case, two different queries with different level of details are prompted into the GAD UI. The first prompt (see Fig. 3c) provides all details and parameters needed for the design of a spur gear, while the second (refer to Fig. 3d) simply requests the component without any engineering details. The results show that in both cases GAD can create the requested item. The outcome for the second query (Fig. 3d) supports that, given the assuming nature of large language models, these systems can still generate useful results (here, CAD models) even with incomplete user input.

Figure 4 showcases the process of designing and producing two spur gears with a 1:2 ratio, printed on a desktop 3D printer using the G-code generated by GAD. This example underlines GAD's utility in rapid prototyping and educational contexts, especially since users can directly link a 3D printer to the system and proceed with the printing process through the UI. Initially, GAD generates the CAD model based on user-defined inputs specified in Fig. 4a. The resulting geometry (see Fig. 4b) is then sliced to be prepared for the manufacturing process (see Fig. 4c). Note that, although this process does not address manufacturing-specific considerations, such as part orientation and support generation, gears

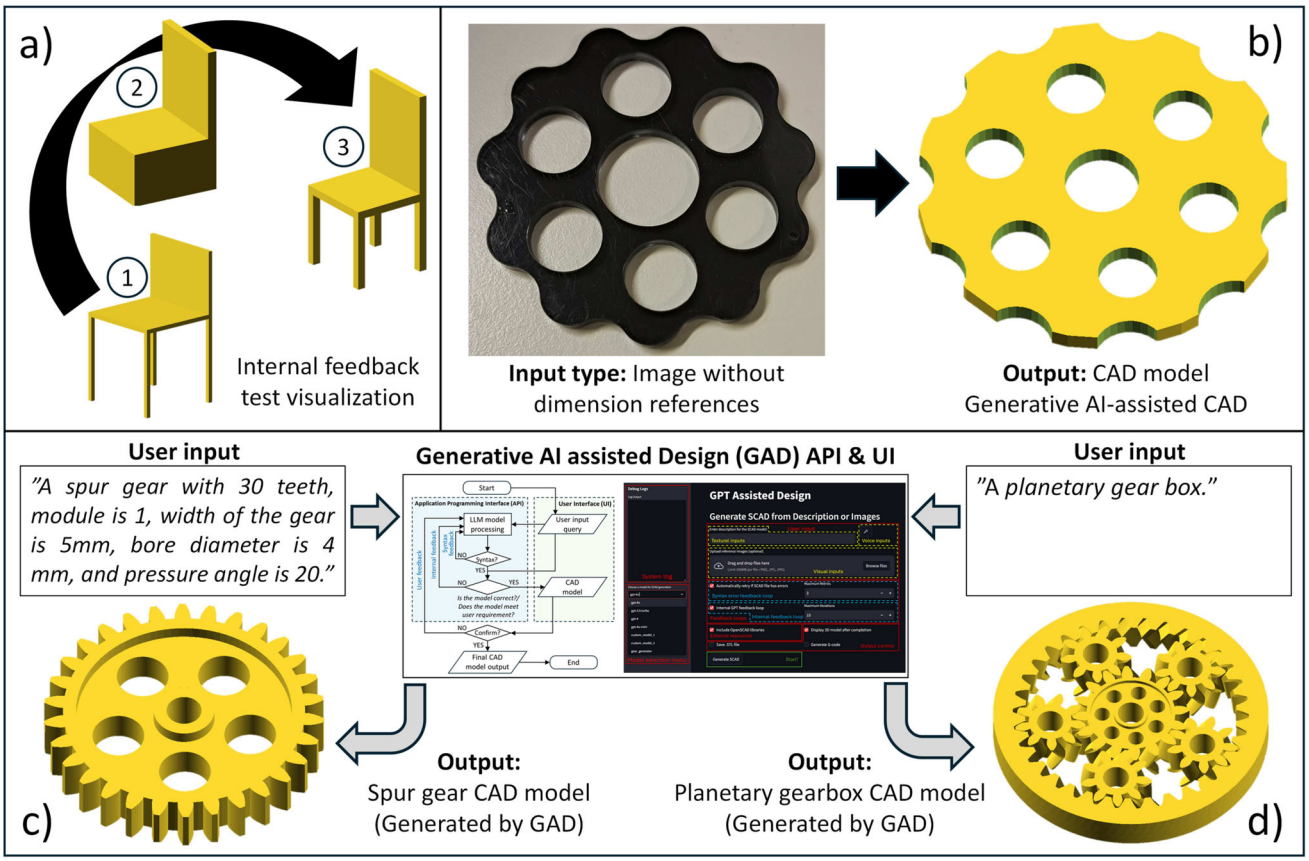

Fig. 3 GAD testing and validation results. **a** Investigation of internal feedback loop on the quality of results, **b** results extracted from GAD in case of only visual inputs, **c** GAD response to a complete user input, and **d** a planetary gear box generated by GAD

are oriented in a way that allows for straightforward production. Finally, Fig. 4d illustrates the printed gears, demonstrating the overall efficiency and accessibility of the process.

Additionally, Table 2 provides a comparative analysis of six key geometric parameters for the gears shown in Fig. 4, including tip diameter, bore diameter, face width, whole

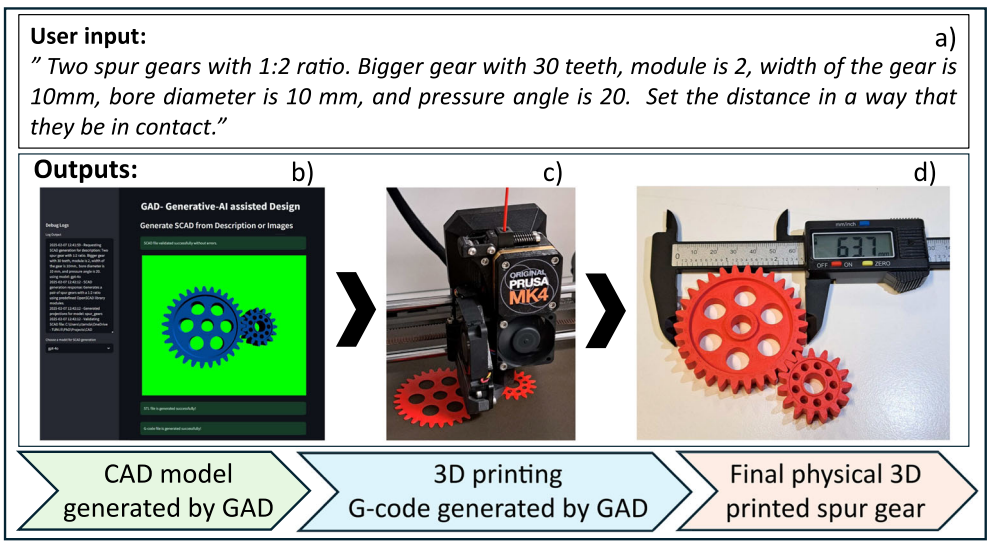

Fig. 4 Design to manufacturing process handled by GAD

Table 2 Design verification measurements of GAD generated CAD and manufactured part

Parameter (Unit)	Designed (CAD)		3D Printed	
	Gear	Pinion	Gear	Pinion
Tip diameter (mm)	63.68	33.98	63.6	33.8
Bore diameter (mm)	10	10	9.7	9.7
Face width (mm)	10	10	9.9	10.1
Whole depth (mm)	4.34	4.34	4.4	4.4
Number of teeth (—)	30	15	30	15
Center distance (mm)	45		44.2	

depth, number of teeth, and the center distance between gears. A direct comparison of the CAD-derived measurements with the user-defined specifications (Fig. 4a) confirms GAD's capability to produce CAD models that closely match user requirements. Minor deviations observed in the final printed parts are attributed to the typical manufacturing tolerances of the desktop 3D printers.

Table 3 presents the performance analysis of various models, e.g., Gemini 2.5 Pro and OlympicCoder, when employed as the processing core in the GAD framework. In this section, the success rate and execution time of these alternative models are examined using the same prompt as in Task 4 (see Fig. 4) across 10 attempts. Although the required SCAD code was successfully generated by Google Gemini 2.5 Pro in 6 out of 10 attempts and OlympicCoder successfully generated the SCAD scripts in all 10 repetitions, the execution time was significantly large at this stage, which could negatively impact the applicability of these alternative models as the primary core for the GAD framework.

4 Conclusion

This paper demonstrates the potential of LLM-driven design systems. It is exemplified by the GAD, capable of creating CAD models and G-code for 3D printing from diverse

Table 3 Performance analysis of alternative models

Model	Attempt	Description	Success	Execution time (s)	Error
Gemini 2.5 Pro Experimental	1	Two spur gears	TRUE	222.8	—
	2	with 1:2 ratio	TRUE	150.9	—
	3	Bigger gear with 30 teeth, module	FALSE	—	SCAD code not generated
	4	is 2, width of the	TRUE	208.4	—
	5	gear is 10 mm bore diameter is	FALSE	—	SCAD code not generated
	6	10 mm, and pressure angle is	FALSE	—	SCAD code not generated
	7	20. Set the	TRUE	17.2	—
	8	distance in a way that they be	TRUE	214	SCAD code not extracted
	9	in contact	TRUE	59	—
	10		FALSE	—	SCAD code not generated
OlympicCoder	1	Two spur gears with 1:2 ratio.	TRUE	144.8	Missing SCAD code elements.
	2	Bigger gear with 30 teeth, module	TRUE	106.2	Missing SCAD code elements.
	3	is 2, width of the	TRUE	190.6	—
	4	gear is 10mm,	TRUE	112.4	—
	5	bore diameter is	TRUE	121.4	—
	6	10 mm, and	TRUE	164.6	—
	7	pressure angle is	TRUE	235.4	—
	8	20. Set the distance in a way	TRUE	111.4	Missing SCAD code elements.
	9	that they be in contact.	TRUE	165	SCAD code not extracted.
	10		TRUE	170.4	—

multimodal data inputs. By unifying AI-driven concept generation with practical manufacturing outputs, GAD enhances the rapid prototyping process and shows significant promise for broader applications in both educational and professional design contexts. Validating tests indicate that GAD can generate CAD models from a variety of input types. Moreover, thanks to the LLM's ability to make reasonable assumptions, GAD continues to perform effectively even when provided with incomplete or underspecified inputs. Through its iterative internal loop, GAD can generate, refine, and adapt CAD models according to user requirements. GAD can also make rapid prototyping easier and more accessible by integrating the design-to-manufacturing process, including CAD modeling, slicing, and manufacturing, into one user interface. A comparison between user-specified and final printed gear parameters shows that the observed deviations arise from the manufacturing process, highlighting the system's reliability in following established design criteria. However, there are some limitations in the current status of the existing LLMs. While GAD can interpret images and replicate general shapes, the fidelity of fine details and precise dimensional accuracy still pose challenges, especially when user input lacks sufficient data, or when the component has geometrical complexities. Additionally, in this study, we compared alternative free mid-sized LLMs, such as Google Gemini 2.5 Pro and OlympicCoder, to assess their performance in generating SCAD code. While both models showed high success rates, their large execution times may limit their effectiveness as the primary core for the GAD framework. These findings underscore the trade-offs between model performance and processing efficiency when choosing a core LLM for CAD-related applications. Future work should therefore focus on improving GAD's ability to extrapolate more accurate dimensional information, better handle complex geometries, and refine designs through advanced error-detection and self-correction mechanisms. Furthermore, although commercial LLMs like GPT-4 simplify research and development efforts, they are costly for large-scale commercial deployment consequently, and future improvements in execution speed are necessary to enhance the practicality of alternative models for scalable applications. Future explorations could seek more cost-effective approaches, whether by employing alternative LLMs or utilizing specialized, locally trained variants designed exclusively for CAD generation purposes. In conclusion, the findings of this paper indicate that language model-driven CAD systems hold significant potential for transforming design processes, offering user-friendly, rapidly iterative, and flexible solutions that can accelerate innovation in engineering and beyond. This work lays the foundation for future exploration of modular, multi-agent architectures and domain-specific model optimization tailored to the full design-to-manufacturing pipeline.

Author Contributions

- Amirmohammad Daareyni: conceived the original idea, developed the final codes, developed the methodology, and wrote the original draft of the manuscript.
- Antti Martikkala: developed the core codes and reviewed and edited the manuscript.
- Hossein Mokhtarian: assisted in developing the methodology and contributed to writing the original draft.
- Iñigo Flores Ituarte: contributed to the methods development and critically reviewed the manuscript.

Funding Open access funding provided by Tampere University (including Tampere University Hospital). This work was supported by the project D2M (346874) Research council of Finland - Academy Research Fellow.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Xu Y, Liuand et al (2021) Artificial intelligence: a powerful paradigm for scientific research. Innovation. 2(4). https://doi.org/10.1016/j.xinn.2021.100179 . Accessed on 29 Jan 2025
2. Russell SJ, Norvig P (2016) Artificial intelligence: a modern approach. Pearson, Upper Saddle River, New Jersey, USA. https://aima.cs.berkeley.edu/
3. Jurafsky D (2000) Speech and language processing. Prentice-Hall
4. OpenAI et al. (2024) GPT-4 technical report. https://doi.org/10.48550/arXiv.2303.08774 . Accessed on 29 Jan 2025
5. Devlin J, Chang M-W, Lee K, Toutanova K (2019) BERT: pre-training of deep bidirectional transformers for language understanding. https://doi.org/10.48550/arXiv.1810.04805. Accessed on 29 Jan 2025
6. Kumar P (2024) Large language models (LLMs): survey, technical frameworks, and future challenges. Artif Intell Rev 57(10):260. https://doi.org/10.1007/s10462-024-10888-y. Accessed on 29 Jan 2025
7. Chen M, et al. (2021) Evaluating large language models trained on code. https://doi.org/10.48550/arXiv.2107.03374. Accessed on 25 Jan 2025

8. Wang Y, Wang W, Joty S, Hoi SC (2021) Codet5: identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv:2109.00859
9. Martikkala A, David J, Ituarte IF (2024) Chatgpt + arduino - a powerful approach for unique use-case tailored sensors. IET Conf Proc 2024:124–131. https://doi.org/10.1049/icp.2024.3496. https://digital-library.theiet.org/doi/pdf/10.1049/icp.2024.3496
10. Sarcar MMM, Rao KM, Narayan KL (2008) Computer aided design and manufacturing. PHI Learning Pvt. Ltd., New Delhi, India. https://shorturl.at/Htf1B
11. Jaisawal R, Agrawal V (2021) Generative design method (GDM) - a state of art. IOP Conf Ser Mater Sci Eng 1104(1):012036. https://doi.org/10.1088/1757-899X/1104/1/012036. Accessed 2025-01-29
12. Brière-Côté A, Rivest L, Maranzana R (2013) 3D CAD model comparison: an evaluation of model difference identification technologies. Comput Aided Des Appl 10(2):173–195. https://doi.org/10.3722/cadaps.2013.173-195. https://www.tandfonline.com/doi/pdf/10.3722/cadaps.2013.173-195. Accessed 29 Jan 2025
13. Daareyni A, Queguineur A, Mokhtarian H, Asadi R, Ituarte IF (2024) Multi-objective optimization-driven design: generative design approach for manufacturing of a train bogie. In: Wang Y-C, Chan SH, Wang, Z-H (eds) Flexible automation and intelligent manufacturing: manufacturing innovation and preparedness for the changing world order. Springer, Cham, pp 48–55. https://doi.org/10.1007/978-3-031-74485-3_6
14. Careri F, Stella L, Khan RHU, Attallah MM (2025) Application of machine learning in additive manufacturing of a novel Al alloy heat exchanger. Int J Adv Manuf Tech. https://doi.org/10.1007/s00170-025-15389-y. Accessed on 01 Apr 2025
15. Kiangala KS, Wang Z (2025) A generative pre-trained transformer industrial bot to improve operators' working experience in a small Industry 5.0 factory. Int J Adv Manuf Tech 136(7):3525–3541. https://doi.org/10.1007/s00170-025-15033-9 . Accessed on 02 Apr 2025
16. Introducing Text-to-CAD (2023). https://zoo.dev/introducing-text-to-cad Accessed on 29 Jan 2025
17. Deng H, Khan S, Erkoyuncu JA (2024) An investigation on utilizing large language model for industrial computer-aided design automation. Procedia CIRP 128:221–226. https://doi.org/10.1016/j.procir.2024.07.049. Accessed on 29 Oct 2024
18. Li X, Sun Y, Sha Z (2024) LLM4CAD: multi-modal large language models for 3d computer-aided design generation. International design engineering technical conferences and computers and information in engineering conference, vol. Volume 6: 36th international conference on Design Theory and Methodology (DTM). pp 006–06015. https://doi.org/10.1115/DETC2024-143740
19. Jones BT, Hähnlein F, Zhang Z, Ahmad M, Kim V, Schulz A (2025) A solver-aided hierarchical language for LLM-driven CAD design. https://doi.org/10.48550/arXiv.2502.09819. Accessed 15 May 2025
20. Li J, Ma W, Li X, Lou Y, Zhou G, Zhou X (2025) CAD-Llama: leveraging large language models for computer-aided design parametric 3D model generation. https://doi.org/10.48550/arXiv.2505.04481. Accessed on 15 May 2025
21. Streamlit (2021) A faster way to build and share data apps. https://streamlit.io/. Accessed on 30 Jan 2025
22. OpenSCAD. https://openscad.org. Accessed on 30 Jan 2025
23. SpeechRecognition: Library for performing speech recognition, with support for several engines and APIs, online and offline. https://github.com/Uberi/speech_recognition#readme. Accessed 30 Jan 2025
24. Slic3r - Open source 3D printing toolbox. https://slic3r.org/. Accessed on 30 Jan 2025
25. kliment (2025) kliment/Printrun. original-date: 2011-05-10T07:41:19Z. https://github.com/kliment/Printrun. Accessed on 12 Feb 2025
26. Gemini 2.5 Pro (2025). https://deepmind.google/technologies/gemini/pro/. Accessed on 08 Apr 2025
27. OlympicCoder 32B (free) - API, Providers, Stats. https://openrouter.ai/open-r1/olympiccoder-32b:free. Accessed on 08 Apr 2025
28. OpenRouter. https://openrouter.ai. Accessed on 08 Apr 2025